Recursión, pilas, colas y diccionarios

Programación: concepto, ejemplo y problema de juez

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1 Recursión sobre un árbol

2 Pilas y colas

3 Diccionarios y conjuntos

4 Problemas para practicar

Compilar, correr y comparar

```
g++ -std=c++17 -Wall -Wextra solucion.cpp -o solucion
./solucion < 1.in > salida.out
diff salida.out 1.out

python3 solucion.py < 1.in > salida.out
diff salida.out 1.out
```

Antes de empezar

Copie la muestra del enunciado en `1.in` y su salida en `1.out`. Si `diff` no imprime nada, las dos salidas coinciden. El juez corre una versión antigua de Python 3: la salida se arma con `print` y `%`, como en los archivos de lectura. Los archivos `lectura_*` están en la carpeta `codigo` de la página de la sesión, en las notas del curso.

Contenido

1 Recursión sobre un árbol
- Concepto
- Ejemplo
- Ejercicio: UVa 839

2 Pilas y colas

3 Diccionarios y conjuntos

4 Problemas para practicar

Lo que cuesta una recursión

Dos casos

El caso base responde sin llamar a nadie. El caso recursivo llama sobre algo más pequeño, y así llega al base.

En el árbol	Lo que dice
Un nodo	una llamada; sus hijos son las llamadas que hace
Los nodos	con trabajo fijo por llamada, el costo total
La altura	los marcos que la pila guarda a la vez: Python corta a los 1000 niveles

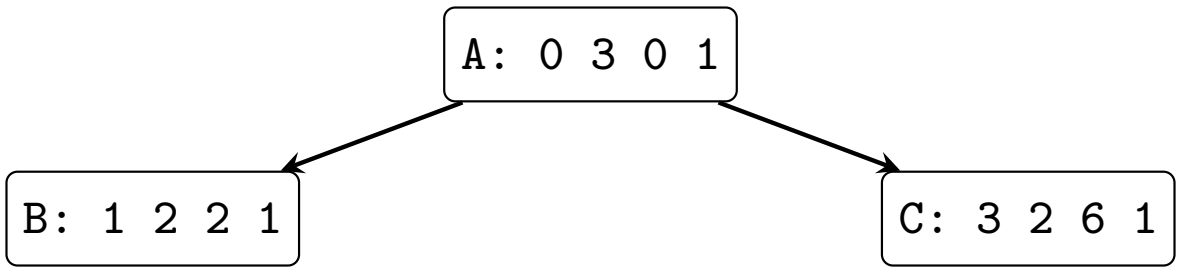
Un móvil de tres barras

```
0 3 0 1
1 2 2 1
3 2 6 1
```

Pregunta

Cada línea es una barra: `Wl Dl Wr Dr`, pesos y distancias. Un peso 0 dice que de ese lado cuelga otra barra, y está en equilibrio si $W_l D_l = W_r D_r$. ¿Se decide la barra de arriba solo con `0 3 0 1`?

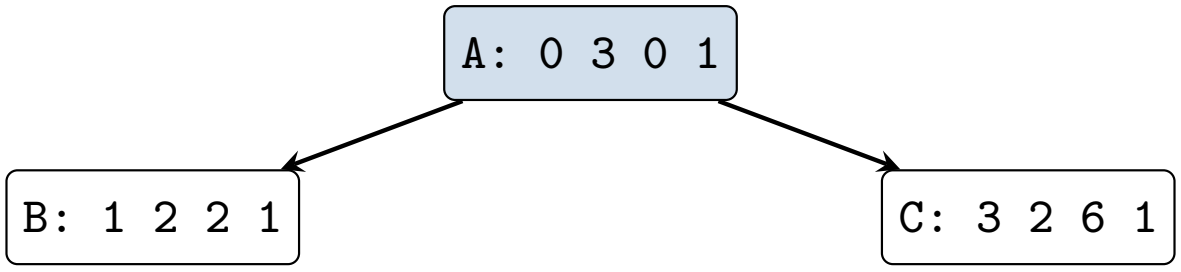
Un móvil de tres barras



Respuesta

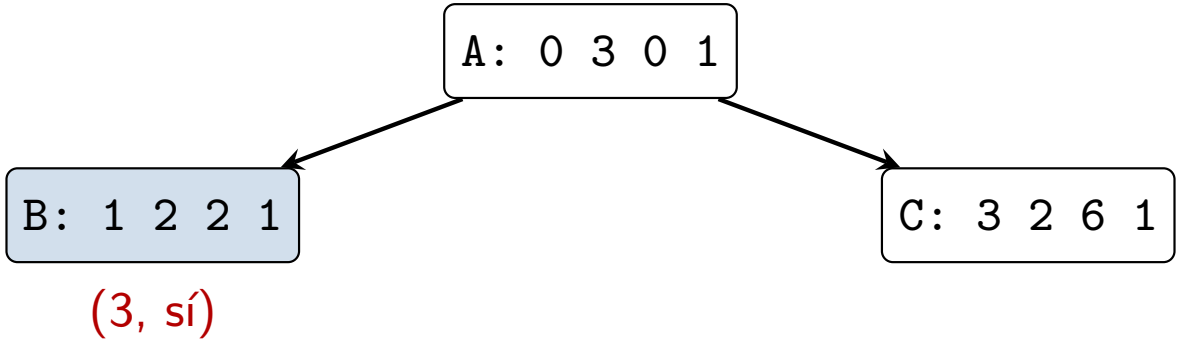
No: $0 \cdot 3 = 0 \cdot 1$ da verdadero siempre. Los pesos de verdad están en las líneas de abajo, así que antes de comparar hay que leer los dos lados. Cada barra es una llamada, y cada peso 0 es un hijo.

Un móvil de tres barras



Paso	Pila	Lee	Devuelve
1	A	0 3 0 1	

Un móvil de tres barras

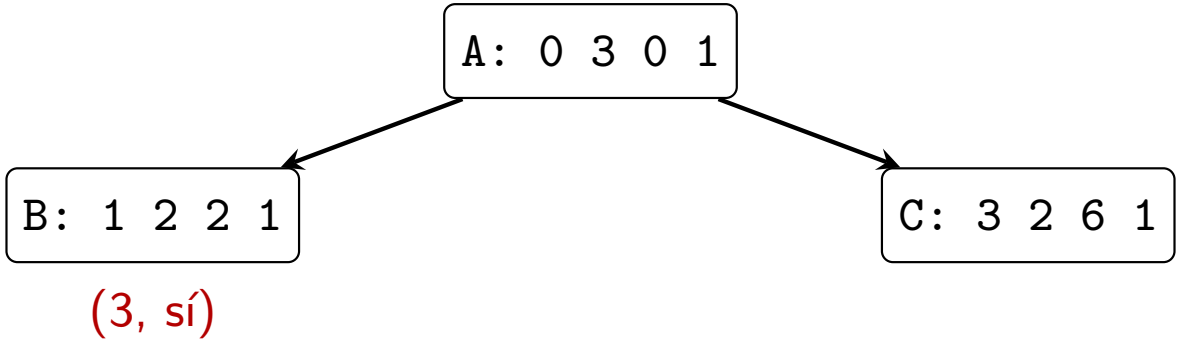


Paso	Pila	Lee	Devuelve
1	A	0 3 0 1	
2	A, B	1 2 2 1	(3, sí)

En el paso 2

B no tiene ceros: $1 \cdot 2 = 2 \cdot 1$. Devuelve lo que pesa y si está en equilibrio.

Un móvil de tres barras

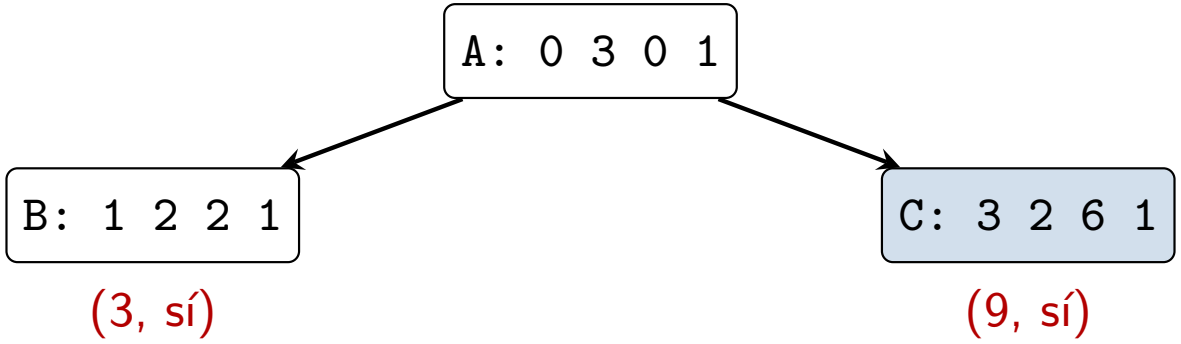


Paso	Pila	Lee	Devuelve
1	A	0 3 0 1	
2	A, B	1 2 2 1	(3, sí)

Pregunta

B ya salió de la pila. ¿Qué línea se lee ahora, y qué devuelve esa llamada?

Un móvil de tres barras

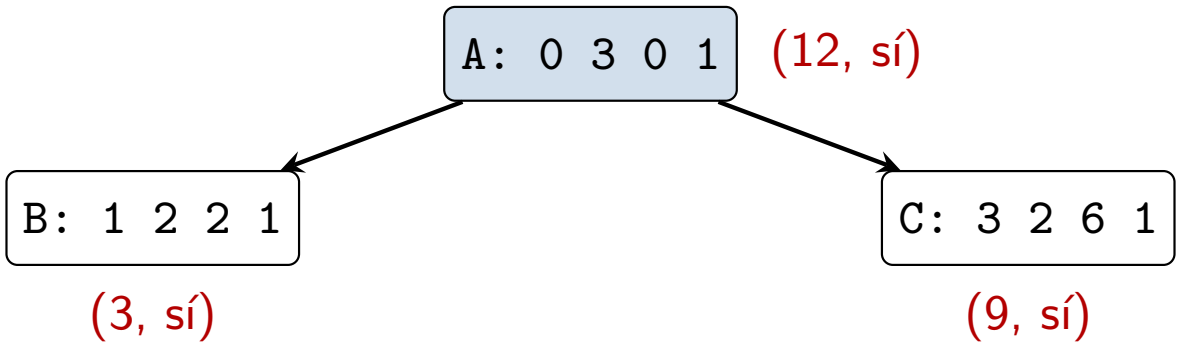


Paso	Pila	Lee	Devuelve
1	A	0 3 0 1	
2	A, B	1 2 2 1	(3, sí)
3	A, C	3 2 6 1	(9, sí)

En el paso 3

La tercera línea, porque A todavía tiene un cero a la derecha: $3 \cdot 2 = 6 \cdot 1$.

Un móvil de tres barras

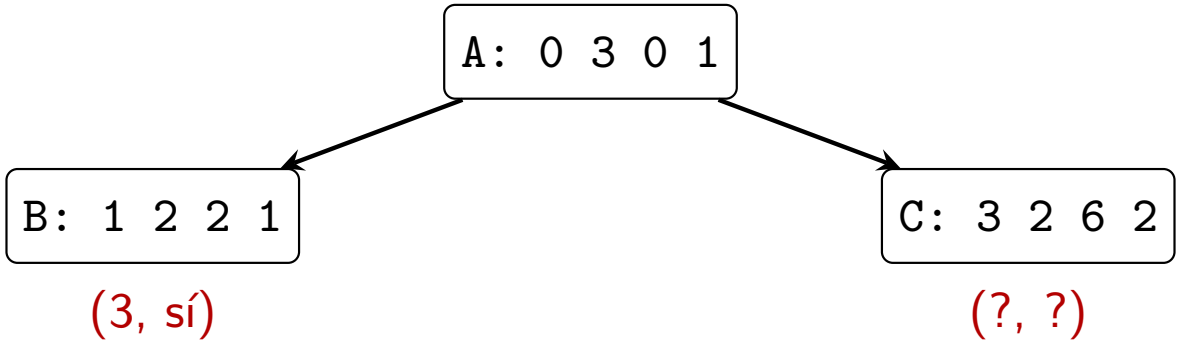


Paso	Pila	Lee	Devuelve
1	A	0 3 0 1	
2	A, B	1 2 2 1	(3, sí)
3	A, C	3 2 6 1	(9, sí)
4	A		(12, sí)

En el paso 4

Con los dos pesos de verdad, $3 \cdot 3 = 9 \cdot 1$. Hubo tres llamadas, una por barra, y a lo más dos marcos en la pila.

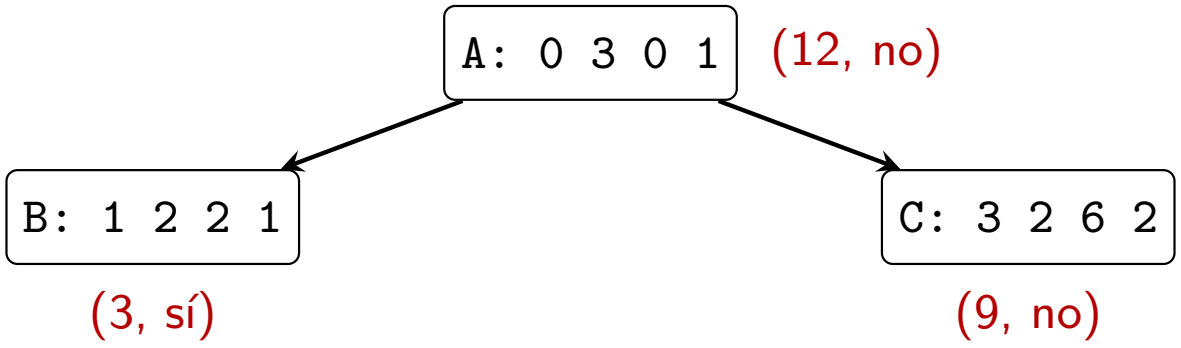
Una barra de abajo que falla



Pregunta

La tercera línea cambia a 3 2 6 2. ¿Qué devuelve C? ¿La barra de arriba sigue en equilibrio? ¿Cuál es la respuesta para todo el móvil?

Una barra de abajo que falla



Respuesta

C devuelve (9, no), porque $3 \cdot 2 \neq 6 \cdot 2$. C sigue pesando 9, así que arriba $3 \cdot 3 = 9 \cdot 1$ se cumple. Aun así el móvil no está en equilibrio: el "no" tiene que subir por el árbol.

UVa 839: Not so Mobile

Enunciado, en corto

Un móvil es una barra con un objeto en cada extremo, o con otro móvil colgando en lugar del objeto. Se pide decidir si todo el móvil está en equilibrio: cada una de sus barras.

Enunciado: `https://onlinejudge.org/external/8/839.pdf`

Envío:

`https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=780`

UVa 839: entrada y salida

```
1

0 2 0 4
0 3 0 1
1 1 1 1
2 4 4 2
1 6 3 2
```

```
YES
```

En palabras

Primero llega el número de casos, con una línea en blanco antes de cada uno. Cada barra es una línea `Wl Dl Wr Dr`, y las de un submóvil siguen a su barra: todo el lado izquierdo y después todo el derecho. Sale `YES` o `NO` por caso, con una línea en blanco entre dos casos.

UVa 839: cómo se lee

```
void leerBarra(int nivel) {
    int wl, dl, wr, dr;
    scanf("%d %d %d %d", &wl, &dl, &wr, &dr);
    for (int i = 0; i < nivel; i++) {
        printf("    ");
    }
    printf("barra: Wl=%d Dl=%d Wr=%d Dr=%d\n", wl, dl, wr, dr);
    if (wl == 0) {
        leerBarra(nivel + 1);
    }
    if (wr == 0) {
        leerBarra(nivel + 1);
    }
}
```

Archivo de partida en C++

`lectura_839.cpp` lee con `scanf` el móvil entero e imprime cada barra, sin resolver nada.

UVa 839: cómo se lee

```
def leer_barra(datos, pos, nivel):
    # lee la barra que empieza en datos[pos] y, si de algún lado cuelga un
    # submóvil, lo lee entero antes de seguir; devuelve dónde sigue la entrada
    wl = int(datos[pos])
    dl = int(datos[pos + 1])
    wr = int(datos[pos + 2])
    dr = int(datos[pos + 3])
    pos = pos + 4
    print("    " * nivel + "barra: Wl=%d Dl=%d Wr=%d Dr=%d" % (wl, dl, wr, dr))
    if wl == 0:
        pos = leer_barra(datos, pos, nivel + 1)
    if wr == 0:
        pos = leer_barra(datos, pos, nivel + 1)
    return pos
```

Archivo de partida en Python

`lectura_839.py` toma la entrada como una lista de palabras, y la función devuelve dónde sigue.

UVa 839: cómo se lee

```
$ python3 lectura_839.py < 1.in
caso 1
barra: Wl=0 Dl=2 Wr=0 Dr=4
    barra: Wl=0 Dl=3 Wr=0 Dr=1
        barra: Wl=1 Dl=1 Wr=1 Dr=1
        barra: Wl=2 Dl=4 Wr=4 Dr=2
    barra: Wl=1 Dl=6 Wr=3 Dr=2
```

La sangría es el árbol

Cada barra sale con la profundidad de su llamada. Las dos de mayor sangría cuelgan de la segunda línea, y la última es el lado derecho de la primera.

A resolver en el computador

Lo que se pide

1. Parta del archivo de lectura, en Python o en C++.
2. Cambie la función para que devuelva lo que hace falta.
3. Quite las impresiones e imprima `YES` o `NO` por caso.
4. Pase la muestra con `diff`.
5. Envíelo al juez.

Contenido

1 Recursión sobre un árbol

2 Pilas y colas
- Concepto
- Ejemplo
- Ejercicio: UVa 12100

3 Diccionarios y conjuntos

4 Problemas para practicar

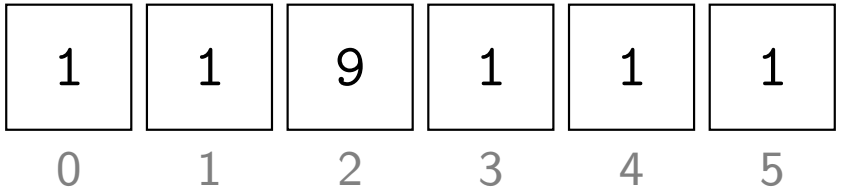
Lo que cuesta cada operación

	Python	C++	Costo
Pila	`list`: `append`, `pop`	`stack`	$O(1)$
Cola	`deque`: `append`, `popleft`	`queue`	$O(1)$
Sacar del frente	`list`: `pop(0)`		$\Theta(n)$

Lo que no da una cola

Una cola solo toca sus dos extremos. Preguntar por lo que hay en el medio obliga a recorrerla, a menos que esa respuesta se lleve aparte.

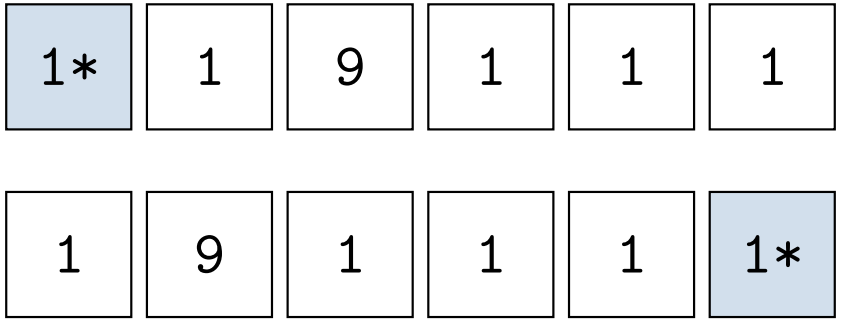
Una cola con marca



Pregunta

Una impresora saca el primer trabajo y, si queda alguno de prioridad mayor, lo manda al final. Si no, lo imprime, y eso toma un minuto. El trabajo propio está en la posición 0. ¿Basta con recordar ese 0?

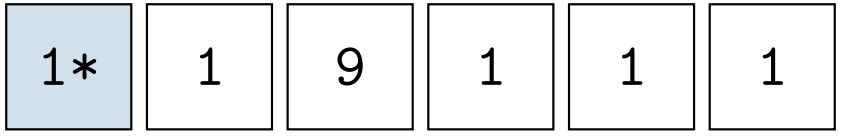
Una cola con marca



Respuesta

No. Tras una rotación, la posición 0 es otro trabajo. La marca tiene que viajar con el trabajo: cada elemento de la cola es un par, la prioridad y si es el propio. En la traza, 1*.

Una cola con marca



Pregunta

Cada vez que sale un trabajo hay que saber si queda uno de prioridad mayor. ¿Hay que recorrer la cola entera?

Una cola con marca

Respuesta

Recorrerla sirve, con $\Theta(n)$ por paso. Las prioridades van de 1 a 9, así que nueve contadores responden en un número fijo de pasos. En `c[p]` va cuántos trabajos de prioridad `p` siguen en la cola, y solo baja al imprimir: rotar no saca a nadie.

Una cola con marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0

En el paso 1

Sale el trabajo propio, y c[9] vale 1: va al final con su marca. No pasa tiempo.

Una cola con marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0
2	1	9 1 1 1 1* 1	5	1	0

Pregunta

Ahora sale el 9. ¿Qué cambia en los contadores y en los minutos?

Una cola con marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0
2	1	9 1 1 1 1* 1	5	1	0
3	9	1 1 1 1* 1	5	0	1

Respuesta

Nada supera al 9: se imprime. c[9] baja a 0 y corre el primer minuto. Las dos rotaciones no costaron nada.

Una cola con marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0
2	1	9 1 1 1 1* 1	5	1	0
3	9	1 1 1 1* 1	5	0	1
4	1	1 1 1* 1	4	0	2

Una cola con marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0
2	1	9 1 1 1 1* 1	5	1	0
3	9	1 1 1 1* 1	5	0	1
4	1	1 1 1* 1	4	0	2
5	1	1 1* 1	3	0	3

Una cola con marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0
2	1	9 1 1 1 1* 1	5	1	0
3	9	1 1 1 1* 1	5	0	1
4	1	1 1 1* 1	4	0	2
5	1	1 1* 1	3	0	3
6	1	1* 1	2	0	4

Una cola con marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0
2	1	9 1 1 1 1* 1	5	1	0
3	9	1 1 1 1* 1	5	0	1
4	1	1 1 1* 1	4	0	2
5	1	1 1* 1	3	0	3
6	1	1* 1	2	0	4
7	1*	1	1	0	5

Resultado

El trabajo marcado se imprime en el minuto 5.

Comparar la prioridad en lugar de la marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0
2	1	9 1 1 1 1* 1	5	1	0
3	9	1 1 1 1* 1	5	0	1
4	1	1 1 1* 1	4	0	2
5	1	1 1* 1	3	0	3
6	1	1* 1	2	0	4
7	1*	1	1	0	5

Pregunta

Alguien guarda solo la prioridad del trabajo propio, 1, y termina cuando se imprime uno de prioridad 1. ¿Qué responde?

Comparar la prioridad en lugar de la marca

Paso	Sale	Cola después	c[1]	c[9]	Minutos
1	1*	1 9 1 1 1 1*	5	1	0
2	1	9 1 1 1 1* 1	5	1	0
3	9	1 1 1 1* 1	5	0	1
4	1	1 1 1* 1	4	0	2
5	1	1 1* 1	3	0	3
6	1	1* 1	2	0	4
7	1*	1	1	0	5

Respuesta

Responde 2, y lo correcto es 5. En el paso 4 se imprime un 1 sin marca: varios trabajos comparten prioridad, y solo la marca distingue el propio.

UVa 12100: Printer Queue

Enunciado, en corto

Una cola de n trabajos, con $1 \leq n \leq 100$, y prioridades de 1 a 9. Dada la posición m del trabajo propio, contada desde 0, se pide el minuto en que termina de imprimirse. Imprimir toma un minuto y mover trabajos no toma tiempo.

Enunciado: `https://onlinejudge.org/external/121/12100.pdf`

Envío:

`https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=3252`

UVa 12100: entrada y salida

```
3
1 0
5
4 2
1 2 3 4
6 0
1 1 9 1 1 1
```

```
1
2
5
```

En palabras

Primero llega el número de casos, hasta 100. Cada caso trae una línea con n y m y otra con las n prioridades, de la primera de la cola a la última. La salida es un número por caso, en su propia línea. El tercer caso es el de la traza.

UVa 12100: cómo se lee, `lectura_12100.cpp`

```cpp
#include <cstdio>
#include <vector>

using namespace std;

int main() {
    int casos;
    scanf("%d", &casos);
    for (int c = 1; c <= casos; c++) {
        int n, m;
        scanf("%d %d", &n, &m);
        vector<int> prioridades(n);
        for (int i = 0; i < n; i++) {
            scanf("%d", &prioridades[i]);
        }
        printf("caso %d: %d trabajos, el propio en la posicion %d\n", c, n, m);
        for (int i = 0; i < n; i++) {
            printf("    posicion %d: prioridad %d\n", i, prioridades[i]);
        }
    }
    return 0;
}
```

UVa 12100: cómo se lee, `lectura_12100.py`

```
import sys


def main():
    datos = sys.stdin.read().split()
    casos = int(datos[0])
    pos = 1
    for c in range(1, casos + 1):
        n = int(datos[pos])
        m = int(datos[pos + 1])
        pos = pos + 2
        prioridades = []
        for i in range(n):
            prioridades.append(int(datos[pos + i]))
        pos = pos + n
        print("caso %d: %d trabajos, el propio en la posicion %d" % (c, n, m))
        for i in range(n):
            print("    posicion %d: prioridad %d" % (i, prioridades[i]))


main()
```

Los dos archivos imprimen n, m y las prioridades, sin resolver nada.

A resolver en el computador

Lo que se pide

1. Parta del archivo de lectura, en Python o en C++.
2. Escriba una función que reciba las prioridades y m, y devuelva el minuto.
3. Imprima solo ese número por caso.
4. Pase la muestra con `diff`.
5. Envíelo al juez.

Contenido

1 Recursión sobre un árbol

2 Pilas y colas

3 Diccionarios y conjuntos
- Concepto
- Ejemplo
- Ejercicio: UVa 156

4 Problemas para practicar

Lo que cuesta un diccionario

	Python: `dict`, `set`	C++: `map`, `set`
Buscar o insertar	$O(1)$ en promedio	$O(\log n)$
Recorrer por clave	`sorted`: $O(n \log n)$	ya va en orden: $O(n)$

Para qué se usa

Un diccionario agrupa: lo que comparte clave cae en la misma casilla, y ahí se lleva una cuenta. Un conjunto guarda solo las claves y dice si una ya está.

Anagramas con una clave

`ROMA` `nube` `amor` `Sol` `mora`

Pregunta

Dos palabras son anagramas si tienen las mismas letras, sin importar el orden ni las mayúsculas. Comparando cada palabra con todas las demás, ¿cuántas comparaciones hay con estas cinco? ¿Y con n?

Anagramas con una clave

`ROMA` `nube` `amor` `Sol` `mora`

Respuesta

Diez pares: $4 + 3 + 2 + 1$. Con n palabras son $n(n-1)/2$, es decir $\Theta(n^2)$ comparaciones.

Anagramas con una clave

`ROMA` `amor` `mora`

Pregunta

Baje cada una a minúscula y ordene sus letras. ¿Qué cadena da cada una?

Anagramas con una clave

`ROMA` $\rightarrow$ `amor` `amor` $\rightarrow$ `amor` `mora` $\rightarrow$ `amor`

Respuesta

Las tres dan `amor`. Dos palabras son anagramas cuando dan la misma cadena, así que esa cadena sirve de clave de un diccionario, y basta contar cuántas palabras caen en cada clave.

Anagramas con una clave

Palabra	Clave	Diccionario después
`ROMA`	`amor`	`amor:1`

Anagramas con una clave

Palabra	Clave	Diccionario después
`ROMA`	`amor`	`amor:1`
`nube`	`benu`	`amor:1 benu:1`

Pregunta

Llega `amor`. ¿Qué pasa con el diccionario?

Anagramas con una clave

Palabra	Clave	Diccionario después
`ROMA`	`amor`	`amor:1`
`nube`	`benu`	`amor:1 benu:1`
`amor`	`amor`	`amor:2 benu:1`

Respuesta

Su clave ya está: la cuenta sube a 2, sin mirar las demás palabras.

Anagramas con una clave

Palabra	Clave	Diccionario después
`ROMA`	`amor`	`amor:1`
`nube`	`benu`	`amor:1 benu:1`
`amor`	`amor`	`amor:2 benu:1`
`Sol`	`los`	`amor:2 benu:1 los:1`

Anagramas con una clave

Palabra	Clave	Diccionario después
`ROMA`	`amor`	`amor:1`
`nube`	`benu`	`amor:1 benu:1`
`amor`	`amor`	`amor:2 benu:1`
`Sol`	`los`	`amor:2 benu:1 los:1`
`mora`	`amor`	`amor:3 benu:1 los:1`

Pregunta

Se imprimen las palabras sin anagrama, tal como llegaron y en orden de código de carácter. ¿Cuáles salen, y en qué orden?

Anagramas con una clave

Palabra	Clave	Diccionario después
`ROMA`	`amor`	`amor:1`
`nube`	`benu`	`amor:1 benu:1`
`amor`	`amor`	`amor:2 benu:1`
`Sol`	`los`	`amor:2 benu:1 los:1`
`mora`	`amor`	`amor:3 benu:1 los:1`

Respuesta

`Sol` y después `nube`: son las dos con cuenta 1, y las mayúsculas van antes que las minúsculas. Las claves solo sirven para contar; lo que se ordena e imprime es la palabra original.

Anagramas con una clave

Palabra	Clave	Diccionario después
`ROMA`	`amor`	`amor:1`
`nube`	`benu`	`amor:1 benu:1`
`amor`	`amor`	`amor:2 benu:1`
`Sol`	`los`	`amor:2 benu:1 los:1`
`mora`	`amor`	`amor:3 benu:1 los:1`

Pregunta

Si la clave no baja a minúscula, ¿qué cambia en la salida?

Anagramas con una clave

Palabra	Clave	Diccionario después
ROMA	AMOR	AMOR:1
nube	benu	AMOR:1 benu:1
amor	amor	AMOR:1 benu:1 amor:1
Sol	Slo	AMOR:1 benu:1 amor:1 Slo:1
mora	amor	AMOR:1 benu:1 amor:2 Slo:1

Respuesta

Sale también ROMA: la salida es ROMA, Sol, nube. Su clave AMOR no coincide con amor.

UVa 156: Ananagrams

Enunciado, en corto

Un diccionario de hasta 1000 palabras de hasta 20 letras. Se piden las palabras sin ningún anagrama en el diccionario, tal como venían escritas y en orden lexicográfico que distingue mayúsculas.

Enunciado: `https://onlinejudge.org/external/1/156.pdf`

Envío:

`https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=92`

UVa 156: entrada y salida

```
ladder came tape soon leader acme RIDE lone Dreis peat
 ScAlE orb eye Rides dealer NotE derail LaCeS drIed
noel dire Disk mace Rob dries
#
```

```
Disk
NotE
derail
drIed
eye
ladder
soon
```

En palabras

Las palabras llegan en líneas de hasta 80 caracteres, con espacios libres alrededor, y la entrada termina con una línea que trae #. La salida lleva una palabra por línea.

UVa 156: cómo se lee, `lectura_156.cpp`

```cpp
#include <cstdio>
#include <string>
#include <vector>

using namespace std;

int main() {
    char leida[100];
    vector<string> palabras;
    int ok = scanf("%s", leida);
    while (ok == 1 && leida[0] != '#') {
        string p = leida;
        palabras.push_back(p);
        ok = scanf("%s", leida);
    }
    printf("%d palabras\n", (int) palabras.size());
    for (int i = 0; i < (int) palabras.size(); i++) {
        printf("    palabra %d: %s\n", i + 1, palabras[i].c_str());
    }
    return 0;
}
```

Lee por palabras hasta el # e imprime cuántas llegaron y cuáles.

UVa 156: cómo se lee, `lectura_156.py`

```
import sys


def main():
    datos = sys.stdin.read().split()
    palabras = []
    i = 0
    while i < len(datos) and datos[i] != "#":
        palabras.append(datos[i])
        i = i + 1
    print("%d palabras" % len(palabras))
    for i in range(len(palabras)):
        print("    palabra %d: %s" % (i + 1, palabras[i]))


main()
```

Archivo de partida

Imprime lo mismo. Leer por palabras evita ocuparse de los espacios y de los saltos de línea.

A resolver en el computador

Lo que se pide

1. Parta del archivo de lectura, en Python o en C++.
2. Escriba la función que calcula la clave de una palabra.
3. Imprima solo las palabras sin anagrama, en el orden que pide el enunciado.
4. Pase la muestra con `diff`.
5. Envíelo al juez.

Contenido

1 Recursión sobre un árbol

2 Pilas y colas

3 Diccionarios y conjuntos

4 Problemas para practicar

Base. UVa 10935: Throwing cards away I

Cómo atacarlo

Un mazo de $n \leq 50$ cartas: se bota la de arriba y la siguiente pasa al fondo. Cada ronda toca los dos extremos del mazo, y una cola hace las dos cosas en tiempo constante.

La trampa: con $n = 1$ no se bota ninguna, y la primera línea es `Discarded cards:` sin espacio final.

Enunciado: `https://onlinejudge.org/external/109/10935.pdf`

Envío:

`https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1876`

Intermedio. UVa 11286: Conformity

Cómo atacarlo

Hasta 10 000 estudiantes, con cinco cursos cada uno. Al ordenar los cinco cursos, dos estudiantes con la misma combinación dan la misma clave de un diccionario.

La trampa: se pide el total de estudiantes, y si varias combinaciones empatan en la mayor popularidad se suman todas.

Enunciado: `https://onlinejudge.org/external/112/11286.pdf`

Envío:

`https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2261`

Exigente. UVa 699: The Falling Leaves

Cómo atacarlo

El árbol llega en preorden y la columna de un hijo depende solo de la de su padre. La función lee un subárbol mientras suma en la columna x y baja a $x-1$ y $x+1$, sin construir el árbol.

La trampa: un árbol puede seguir en la línea siguiente, y la línea en blanco va también después del último caso.

Enunciado: `https://onlinejudge.org/external/6/699.pdf`

Envío:

`https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=640`